

Assignment 3 - Frontend Application

(React.js, Connected to Backend)

In this assignment you will build the React frontend application for your final project. The content, pages, data, and design should match your team's specific project idea, but all teams must implement the required components below. All components must be connected to the backend API you built in Assignment 2.

Your goal is to finish with a frontend that:

- starts and runs locally.
- connects directly to your backend REST API.
- handles client-side routing.
- accepts user input with basic validation.
- displays dynamic data from the backend using reusable components and tables.

Learning goals

- Create a React application with a clean project structure.
- Design REST API integration using **fetch** or **axios**.
- Manage component state (loading, error, data).
- Build reusable UI components (Cards, Tables, Forms).
- Validate input and handle errors cleanly on the client side.

Technical requirements

- **Framework:** React.js (using **create-react-app**).
- **Data Fetching:** Fetch API or Axios.
- **Routing:** React Router.
- **Port:** 3000 by default.
- **Base URL:** `http://localhost:3000` (The backend server from Assignment 2).

Pages and Components

1. Login Page

Create a login page connected to the backend.

- **Requirements:** Email input, Password input, Login button.
- **Validation:** Email is required (valid format), Password is required (at least 6 characters).
- **Backend Endpoint:** `POST /api/auth/login`.
- **Frontend must:** Send login request, show loading state, show error message if login fails, and redirect after successful login.

2. Navbar & Layout

Create a navbar displayed after login.

- **Requirements:** Project name/logo, Navigation links, Settings link, Logged-in user name, and Logout button.
- **Backend Endpoints:** `GET /api/users/me`, `POST /api/auth/logout`.
- **Frontend must:** Display user information from backend and handle logout correctly.

3. Footer

Create a footer shown on all main pages.

- **Requirements:** Project/team name, Year, and short project description or slogan.

4. Settings Page

Create a settings page connected to the backend.

- **Requirements:** At least 3 editable settings (e.g., Username, Email, Theme preference).
- **Backend Endpoints:** GET `/api/settings`, PUT `/api/settings`.
- **Frontend must:** Load current settings, display in a form, validate fields, and show loading/success/error states.

5. Dashboard / Home Page

Create the main page of the project (e.g., Products, Events, Tasks).

- **Backend Endpoints:** e.g., GET `/api/products`.
- **Frontend must:** Fetch data from backend, display loading state, and handle empty states.

6. Reusable Card Component

Create a reusable component that displays backend data.

- **Requirements:** Receive data using props and match the project idea.
- **Usage:** Component must be reused at least 3 times.

7. Data Table Component (Required)

Create at least one `table` element to present your structured data.

- **Requirements:** The table must display one or more data structures stored in your backend (partial or full).
- **Frontend must:** Dynamically map over an array of data retrieved from the backend to generate rows.

Required project structure

Organize your React project following this structure:

- `src/components/` - Reusable UI elements (Navbar, Footer, Card, Table).
- `src/pages/` - Main route views (Login, Dashboard, Settings).
- `src/services/` - API communication logic.
- `src/App.js` - Main entry point and routing configuration.

Documentation requirements

Create a `README.md` file that explains how to run and test your frontend.

- How to install dependencies: `npm install`.
- How to start the server: `npm start`.
- API Base URL: Explicitly state `http://localhost:3000` as the target.

What you must submit

Submit a zip containing:

- Frontend source code (excluding `node_modules`).
- `README.md` (run instructions).
- Screenshots showing the application running (Login, Dashboard, Table, Settings).

Summary

By the end of this assignment you should have a working frontend React application that communicates directly with your backend API and effectively displays your mock data.